

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias

Analizador Sintáctico

Triton GPU Kernel

TC3002B — Computer Science Advanced Applications Development
Módulo #3: Compiladores — Fase de Análisis Sintáctico

Equipo: 11

Cristóbal Camarena Hernández — A01642653

Josué Galindo Gutiérrez — A01637983

Gandhi Emmanuel Valdez Huerta — A01625738

Diego Alberto Estrada López — A01638657

Profesor: Dr. Salvador Hinojosa

Fecha de entrega: Junio 2026

Guadalajara, Jalisco, México

Resumen

Resultados: el analizador sintáctico `triton_parser` obtiene **12/12 PASS** en la suite unitaria, **200/200 (100 %)** en las suites JSONL (`curated_100` + `adversarial_100`) y **58/70 PARSE_OK (82.9 %)** en el benchmark TRITONHEAL. La métrica principal de validación es JSONL: **100 %** de kernels válidos aceptados y **100 %** de casos inválidos rechazados, sin falsos positivos ni falsos negativos. En el benchmark, 2 rechazos son mutaciones sintácticas sembradas; 10 adicionales corresponden a archivos con código suelto en columna 0 (inválido en Python). Este reporte documenta la CFG, el diseño yacc/Flex, las tablas de símbolos, los mensajes de error y la evidencia experimental.

Índice

1. Introducción	1
1.1. Resumen	1
1.2. Notación	1
1.3. Justificación de herramientas	1
1.4. Trazabilidad a la fase léxica	1
2. Análisis	1
2.1. Entregables del reto (Sección IV, documento de evaluación)	1
2.2. Requerimientos funcionales	2
2.3. CFG (resumen)	2
2.4. Pasos para obtener la gramática	3
2.5. Simplificaciones justificadas	3
2.6. Modificaciones al analizador léxico	3
2.7. Patrones léxicos: <code>BAD_ID</code>	4
2.8. Trazabilidad token léxico $\rightarrow$ yacc	4
3. Diseño	4
3.1. Arquitectura del sistema	4
3.2. Integración lex $\leftrightarrow$ yacc	4
3.3. Diseño de la especificación yacc	5
3.4. Tabla de símbolos del parser (diseño)	5
3.5. Mensajes de error (diseño)	5
3.6. Trazabilidad diseño $\rightarrow$ análisis	5
4. Implementación	5
4.1. Estructura del proyecto	5
4.2. Compilación y ejecución	5
4.3. Fragmento: detección <code>BAD_ID</code> (lexer)	6
4.4. Fragmento: integración de indentación (lexer)	6
4.5. Fragmento: reglas principales (yacc)	7
4.6. Fragmento: tabla de símbolos	8
4.7. Fragmento: manejo de errores	10
4.8. Ejemplo de salida del parser (entregable 5)	10
5. Verificación y Validación	11
5.1. Resultados experimentales	11
5.2. Resumen cuantitativo	11
5.3. Suites JSONL (<code>make jsonl-test</code>)	11
5.4. Interpretación de los resultados	12
5.5. Gráfica de resultados (Figura 2)	12

5.6.	Detalle de rechazos en el benchmark	14
5.7.	Modelo de pruebas	14
5.8.	Suite unitaria (<code>make test</code>)	14
5.9.	Evidencia: salida exitosa (<code>ejemplo_1.tri</code>)	14
5.10.	Evidencia: error sintáctico (<code>def_sin_dos_puntos.tri</code>)	15
5.11.	Evidencia: error léxico <code>BAD_ID</code> (I6)	15
5.12.	Reproducibilidad	15
5.12.1.	Prerrequisitos	15
5.12.2.	Pasos desde cero (PC nueva)	16
5.12.3.	Comando único (regeneración completa)	16
5.12.4.	Salidas verificables	16
5.12.5.	Consulta sin recompilar	16
5.12.6.	Notas por sistema operativo	16
6.	Plan de Trabajo	17
6.1.	Distribución de responsabilidades	17
A.	Gramática libre de contexto completa	17
A.1.	Convenciones y notación	17
A.2.	Símbolo inicial y conteo	18
A.3.	Terminales del analizador	18
A.3.1.	Terminales que aparecen en producciones	18
A.3.2.	Terminales declarados y no usados en reglas	18
A.4.	Precedencia y asociatividad (<code>yacc</code>)	19
A.5.	Inventario de no terminales (36)	19
A.6.	Notas semánticas ligadas a la CFG (fuera de producciones)	20
A.7.	Desglose detallado: no terminal <code>expr</code> (16 alternativas)	20
A.8.	Desglose detallado: <code>subscript</code> (5 alternativas)	21
A.9.	Desglose detallado: <code>block_stmt</code> (3 alternativas)	21
A.10.	Listado completo de producciones (P-001 a P-091)	21
A.11.	Implementación <code>yacc</code> (referencia cruzada)	23
A.12.	Trazabilidad y mantenimiento	28
B.	Referencias	28

Índice de figuras

1.	Pipeline del analizador sintáctico.	4
2.	Resultados del analizador sintáctico (benchmark 70 kernels). Verde = <code>PARSE_OK</code> ; naranja/rojo = rechazo (mutaciones <code>k_0040/k_0058</code> o código en columna 0). La validación JSONL 200/200 no se grafica aquí; ver Tabla 6.	13

Índice de cuadros

1.	Simplificaciones de la CFG.	3
2.	Caracteres invalidos en identificadores y patron Flex.	4
3.	Correspondencia Token ID (reporte léxico) y <code>yacc</code> .	4
4.	Esquema de entradas en <code>syntab.c</code> .	5
5.	Cuadro resumen de resultados (corridas locales reproducibles con <code>make test</code> , <code>make jsonl-test</code> y <code>make benchmark</code>).	11
6.	Resultados JSONL (métrica principal del proyecto).	11

7.	Clasificación de los 12 rechazos del benchmark TRITONHEAL.	14
8.	Casos de prueba del equipo (12 ejecuciones).	14
9.	Software necesario para reproducir compilación, pruebas, gráfica e informe. . . .	15
10.	Cronograma de la fase sintáctica (Equipo 11).	17
11.	Resumen formal de la gramática implementada.	18
12.	Terminales usados en al menos una producción de la CFG.	18
13.	Directivas <code>%left</code> / <code>%nonassoc</code> / <code>%right</code> en <code>parser.y</code> (L71–78).	19
14.	Catálogo de no terminales: número de alternativas e intervalo en <code>parser.y</code>	19
15.	Acciones en C asociadas a reglas seleccionadas (no son parte de la CFG).	20

1. Introducción

1.1. Resumen

Este documento presenta el análisis, diseño, implementación y verificación del **analizador sintáctico** para kernels **Triton GPU** escritos en archivos `.tri`. La fase sintáctica es la segunda etapa del compilador del reto académico: consume la secuencia de tokens producida por el analizador léxico (Flex) del Equipo 11, construye una estructura conforme a una gramática libre de contexto (CFG) implementada con **yacc/Bison**, actualiza tablas de símbolos y reporta errores sintácticos.

El entregable incluye: (i) gramática documentada (Apéndice A, CFG completa); (ii) tablas de símbolos; (iii) implementación `triton_parser`; (iv) mensajes de error; (v) **resultados experimentales** con énfasis en la Sección 5.1: **100 %** suite unitaria (12/12), **100 %** suites JSONL (200/200), benchmark TRITONHEAL (58/70) y gráfica de los 70 kernels.

1.2. Notación

- **CFG**: gramática libre de contexto en notación BNF; producciones $A \rightarrow \alpha$.
- **Token ID**: identificador numérico del Cuadro 1 del reporte léxico (100–999).
- **INDENT/DEDENT** (910/911): tokens sintéticos derivados de `WS_COUNT` (900) para modelar indentación Python.
- **PARSE_OK**: análisis sintáctico exitoso con impresión de tabla de símbolos del parser.

1.3. Justificación de herramientas

Se utiliza **C** con **Flex** y **yacc**, conforme a las restricciones del curso. No se emplean librerías CFG de alto nivel ni `ast` de Python para el parser formal.

1.4. Trazabilidad a la fase léxica

Los IDs de token se preservan en `%token` de `parser.y` (p. ej. `TOKEN_IDENTIFIER = 100`). El archivo `lexico_equipo_11_scanner.l` es la base del scanner integrado; el modo `-t` reproduce la salida del reporte léxico (TC-01–TC-04).

2. Análisis

2.1. Entregables del reto (Sección IV, documento de evaluación)

1. Gramática correcta y no ambigua para kernels Triton, con explicación de decisiones.
2. Gestión de tablas de símbolos durante el análisis sintáctico.
3. Implementación del parser con yacc.
4. Mensajes de error sintácticos.
5. Ejemplos de salida del parser.

2.2. Requerimientos funcionales

- Reconocer programas `.tri`: imports, `@triton.jit`, `def`, cuerpos indentados.
- Sentencias: asignación (`ident_stmt`, anotada `id : tipo =`), `if/elif/else`, `while`, `for/in`, `return`, `pass`.
- Expresiones: literales, operadores (500), operador `is`, expresión condicional (`expr if expr else expr`), llamadas `tl.*`, subíndices `[: , None]`, argumentos con nombre (`mask=`, `dtype=`).
- Validación de línea: rechazo de múltiples sentencias en la misma línea (`lexer + validate_stmt_line_end`).
- Indentación: DEDENT automático cuando una línea comienza en columna 0 tras un bloque indentado.
- Rechazar sintaxis inválida con `SYNTAX_ERROR` en español.
- Actualizar `syntab`: imports, funciones, parámetros, variables.

2.3. CFG (resumen)

La gramática formal completa (**36 no terminales**, **105 alternativas** numeradas P-001...P-105, inventario en Tabla 14 del Apéndice A) se documenta en el Listado 11 y el bloque yacc Listado 12. El siguiente extracto resume las construcciones principales para lectura rápida en el análisis.

Listing 1: CFG resumida del lenguaje `.tri` (Triton kernel subset)

```
1 program      -> opt_newlines top_block opt_newlines
2 top_block    -> top_block top_line | epsilon
3 top_line     -> stmt | stmt NEWLINE
4
5 stmt         -> import_stmt | def_stmt | assign_stmt | ident_stmt | if_stmt
6              | while_stmt | for_stmt | return_stmt | pass_stmt | expr_stmt
7
8 ident_stmt   -> TOKEN_IDENTIFIER ident_op
9 ident_op     -> TOKEN_ASSIGN expr | ident_suffix
10
11 block_stmt   -> stmt TOKEN_NEWLINE | stmt /* validate_stmt_line_end */
12
13 import_stmt  -> KW_IMPORT import_name opt_as
14 import_name  -> ID | import_name '.' ID
15
16 opt_triton_decorator -> epsilon | '@' ID '.' ID opt_newlines
17
18 def_stmt     -> opt_triton_decorator KW_DEF ID '(' param_list_opt ')' ':' suite
19
20 suite        -> simple_stmt
21              | opt_newlines INDENT block_body DEDENT
22
23 expr         -> literal | ID ident_suffix
24              | expr OP expr | expr KW_IS expr
25              | expr KW_IF expr KW_ELSE expr
26              | expr '[' subscript_list ']'
27              | '(' arg_list_opt ')' | call_expr | unary_expr
28
29 subscript    -> expr | NONE | ':' opt_expr | opt_expr ':' opt_expr
30              | opt_expr ':' opt_expr ':' opt_expr
31
32 call_expr    -> ID '(' arg_list_opt ')'
33              | ID member_chain '(' arg_list_opt ')'
34
35 arg          -> expr | ID '=' expr
```

2.4. Pasos para obtener la gramática

1. Inventario de tokens del reporte léxico (Cuadro 1).
2. Identificación de estructuras Triton en 70 kernels de referencia.
3. Borrador de producciones para bloques indentados (INDENT/DEDENT).
4. Incorporación de expresiones, llamadas y subíndices tensoriales.
5. Resolución de ambigüedades con precedencias yacc y pruebas iterativas.

2.5. Simplificaciones justificadas

Cuadro 1: Simplificaciones de la CFG.

Tipo	Decisión
Unitarias	Jerarquía de expresiones colapsada en <code>expr</code> con <code>%left</code> .
ε	Solo en listas opcionales, <code>else</code> y líneas vacías iniciales.
Símbolos inútiles	Sin <code>class</code> , <code>try</code> , <code>switch</code> (no usados en kernels).
Indentación	<code>WS_COUNT</code> $\rightarrow$ pila $\rightarrow$ INDENT/DEDENT.
Comentarios	Patrón <code>#(\^)\n]*</code> para no consumir <code>)</code> en la misma línea.

2.6. Modificaciones al analizador léxico

- Integración con `y.tab.h`; `yylex()` retorna códigos de token.
- Tokens sintéticos 910/911 (no en Cuadro 1 original; documentados en este reporte).
- Sin nuevos IDs en el Cuadro 1 oficial; trazabilidad numérica conservada.
- **Detección BAD_ID** (heredada del analizador léxico del equipo en <https://github.com/G4ndhiV/Analizador-lexico>): identificadores que contienen caracteres prohibidos (\$, acento grave, barra invertida, ?, !) se reconocen con una regla Flex dedicada *antes* que ID, emitiendo `LEXICAL_ERROR` (999) sin fragmentar el lexema.
- **Múltiples sentencias en la misma línea**: el wrapper `yylex()` detecta patrones inválidos (`pass pass, return 5 6, x = 5 y = 6`, etc.) y marca `syntax_error_seen`.
- **DEDENT en column 0**: si un identificador inicia una línea sin espacios previos y la pila de indentación está activa, se emite `TOKEN_DEDENT` antes del token (cierre de bloque Python).

2.7. Patrones léxicos: BAD_ID

Cuadro 2: Caracteres invalidos en identificadores y patron Flex.

Definición	Regex	Comportamiento
BAD_ID_CHAR	<code>[\$\?!]</code>	Conjunto de símbolos no permitidos en nombres.
BAD_ID	ver <code>lexer/lexico_equipo_11_scanner.1</code>	Secuencias con al menos un carácter inválido; prioridad sobre ID.
ID	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Identificadores válidos (tabla 100).

Mensaje emitido: `LEXICAL_ERROR: ... (identificador con caracteres invalidos)`. El driver marca `lexical_error_seen` y termina con código de salida 2, sin reportar `PARSE_OK`.

2.8. Trazabilidad token léxico $\rightarrow$ yacc

Cuadro 3: Correspondencia Token ID (reporte léxico) y yacc.

ID	Nombre	yacc
100	IDENTIFIER	<code>TOKEN_IDENTIFIER</code>
101	KEYWORD	<code>KW_def</code> , <code>KW_if</code> , ...
200–400	Literales	<code>TOKEN_INTEGER</code> , etc.
500	OPERATOR	<code>TOKEN_OPERATOR</code>
600	DELIMITER	<code>TOKEN_DELIMITER</code> o literales <code>() [] : ,</code>
900	WS_COUNT	convertido a <code>INDENT/DEDENT</code>
950	NEWLINE	<code>TOKEN_NEWLINE</code>
910/911	(sintéticos)	<code>TOKEN_INDENT</code> , <code>TOKEN_DEDENT</code>

3. Diseño

3.1. Arquitectura del sistema

El diseño sigue el modelo en capas del compilador: **entrada** $\rightarrow$ **scanner** $\rightarrow$ **parser** $\rightarrow$ **acciones semánticas** (`syntab` + salida).

`.tri` $\rightarrow$ Flex (`lexer/`) $\rightarrow$ yacc (`parser.y`) $\rightarrow$ `Syntab` + `stdout/stderr`

Figura 1: Pipeline del analizador sintáctico.

3.2. Integración `lex` $\leftrightarrow$ `yacc`

1. Bison genera `y.tab.c/h` con códigos de token y tipos `YYSTYPE`.
2. Flex incluye `y.tab.h`; `yyflexlex()` reconoce patrones; `yylex()` inyecta `INDENT/DEDENT`.
3. `yylval.i` transporta índices de tabla léxica; `yylval.s` transporta lexemas de operadores y delimitadores.
4. `driver.c` invoca `yyparse()` y, si no hay errores, imprime `PARSE_OK` y tablas.

3.3. Diseño de la especificación yacc

- **Regla inicial:** `program` con bloque superior y `newlines` opcionales.
- **Bloques:** suite usa `INDENT/DEDENT` para cuerpos de función y ramas `if`.
- **Triton:** `opt_triton_decorator` reconoce `@triton.jit` en líneas separadas del `def`.
- **Expresiones:** `member_chain` para `tl.load`; `subscript_list` para indexación tensorial.
- **Precedencia:** `%left` para operadores; `%nonassoc KW_ELSE` para *dangling else*.

3.4. Tabla de símbolos del parser (diseño)

Cuadro 4: Esquema de entradas en `syntab.c`.

Tipo	Campos	Inserción
Import	módulo, alias, línea	<code>import_stmt</code>
Función	nombre, línea, aridad, <code>triton_jit</code>	cierre de <code>def_stmt</code>
Parámetro	función, nombre, posición, anotación	cada <code>param</code>
Variable	nombre, ámbito, línea	<code>assign_stmt</code> con <code>=</code>

3.5. Mensajes de error (diseño)

- **Léxico:** `LEXICAL_ERROR: token_id=<999>line=<n>lexeme="..."`.
- **Sintáctico:** `SYNTAX_ERROR: line=<n>near=" message='...' vía yyerror()`.

3.6. Trazabilidad diseño → análisis

Cada requisito del análisis (Sección 2) se implementa en un módulo: CFG en `parser.y`, indentación en `lexer/`, `syntab` en `syntab.c`, errores en `yyerror`, salidas en `driver.c`.

4. Implementación

4.1. Estructura del proyecto

```
analizador-sintactico/  
  lexer/lexico_equipo_11_scanner.l  
  parser/parser.y, syntab.c, syntab.h  
  src/driver.c  
  build/triton_parser  
  tests/{valid,invalid}/  results/{parser,benchmark}/  
  figures/benchmark_parse_results.pdf
```

4.2. Compilación y ejecución

Instrucciones completas para una PC nueva (clonado, dependencias, salidas): Sección 5.12 y `README.md` en GitHub.

```
git clone https://github.com/G4ndhiV/Analizador-sint-ctico.git  
cd Analizador-sint-ctico  
make check-deps    # opcional  
make build
```

```

make test          # 12 casos unitarios
make jsonl-test    # 200 casos JSONL (curated + adversarial)
make benchmark     # 70 kernels en benchmark/kernels/
make figures
make pdf           # requiere tectonic o latexmk
./build/triton_parser archivo.tri
./build/triton_parser -t archivo.tri  # modo solo lexico

```

4.3. Fragmento: detección BAD_ID (lexer)

Misma política que el repositorio léxico del equipo (Analizador-lexico): la regla BAD_ID tiene precedencia sobre ID para no tokenizar por partes un nombre inválido.

Listing 2: Definiciones Flex BAD_ID_CHAR y BAD_ID.

```

1      paren_depth--;
2  } else if (ch == '[') {
3      bracket_depth++;
4  } else if (ch == ']' && bracket_depth > 0) {
5      bracket_depth--;

```

Listing 3: Acción léxica: error por identificador con caracteres invalidos.

```

1
2 {FLOAT} {
3     int idx = add_to_table(float_table, &float_count, yytext);
4     at_line_start = 0;
5     if (lexer_debug_mode) {
6         emit_token_ref(TOKEN_FLOAT, "FLOAT", yytext, idx);
7     } else if (parser_mode) {

```

4.4. Fragmento: integración de indentación (lexer)

Listing 4: Detección de múltiples sentencias y bol_dedent_token (extracto).

```

1      return tok;
2  }
3
4  static void defer_token(int tok) {
5      deferred_tok = tok;
6      deferred_yylval = ylval;
7  }
8
9  static int lexer_multistmt_error(int tok) {
10     if (!parser_mode || line_no != lex_prev_line) {
11         return 0;
12     }
13     if (tok == KW_PASS && lex_prev == KW_PASS) {
14         return 1;
15     }
16     if (tok == TOKEN_INTEGER && lex_prev == KW_PASS) {
17         return 1;
18     }
19     if (tok == TOKEN_INTEGER && lex_prev == TOKEN_INTEGER && lex_prev2 ==
20         KW_RETURN) {
21         return 1;
22     }
23     if (tok == TOKEN_IDENTIFIER && lex_prev == TOKEN_INTEGER &&
24         saw_assign_this_line) {

```

```

24     }
25     return 0;
26 }
27
28 static void lexer_note_token(int tok) {
29     if (!parser_mode || tok == 0 || tok == TOKEN_NEWLINE || tok == TOKEN_COMMENT
30         ||
31         tok == TOKEN_INDENT || tok == TOKEN_DEDENT) {
32         return;
33     }
34     lex_prev2 = lex_prev;
35     lex_prev = tok;
36     lex_prev_line = line_no;
37 }
38
39 static int bol_dedent_token(void) {
40     if (!parser_mode || !at_line_start || line_has_ws || indent_depth <= 1) {
41         return 0;
42     }
43     if (paren_depth > 0 || bracket_depth > 0) {
44         return 0;
45     }
46     return handle_indent(0, INDENT_KIND_UNKNOWN);
47 }
48
49 int lexer_peek_fresh_line(void) {
50     if (peek_valid) {
51         return peek_fresh_line;
52     }
53     return fresh_line || token_is_fresh;
54 }
55
56 static int push_pending(int tok) {
57     if (pending_count >= (int)(sizeof(pending_tokens) / sizeof(pending_tokens
58         [0]))) {
59         return -1;

```

4.5. Fragmento: reglas principales (yacc)

Listing 5: Producciones program, stmt, def_stmt (extracto).

```

1
2 %left TOKEN_NEWLINE
3 %left KW_OR
4 %left KW_AND
5 %nonassoc KW_NOT
6 %left TOKEN_OPERATOR
7 %nonassoc KW_ELIF
8 %nonassoc KW_ELSE
9 %right KW_IF
10
11 %start program
12
13 %%
14
15 program
16     : opt_newlines top_block opt_newlines
17     { parse_ok = 1; }
18     ;
19
20 opt_newlines
21     : /* empty */

```

```

22     | opt_newlines TOKEN_NEWLINE
23     ;
24
25 top_block
26     : /* empty */
27     | top_block opt_newlines top_line
28     ;
29
30 top_line
31     : import_stmt
32     | import_stmt TOKEN_NEWLINE
33     | def_stmt
34     | def_stmt TOKEN_NEWLINE
35     ;
36
37 stmt
38     : import_stmt
39     | def_stmt
40     | assign_stmt
41     | ident_stmt
42     | if_stmt
43     | while_stmt
44     | for_stmt
45     | return_stmt
46     | pass_stmt
47     | expr_stmt
48     ;
49
50 import_stmt
51     : KW_IMPORT import_name opt_as
52     {
53         symtab_add_import($2, $3 ? $3 : "", line_no);
54     }
55     ;
56
57 import_name
58     : TOKEN_IDENTIFIER
59     {
60         $$ = strdup(id_text($1));
61     }
62     | import_name '.' TOKEN_IDENTIFIER
63     {
64         size_t len = strlen($1) + strlen(id_text($3)) + 2;
65         char *buf = (char *)malloc(len);
66         snprintf(buf, len, "%s.%s", $1, id_text($3));
67         free($1);
68         $$ = buf;
69     }
70     ;
71
72 opt_as
73     : /* empty */ { $$ = NULL; }
74     | KW_AS TOKEN_IDENTIFIER { $$ = (char *)id_text($2); }
75     ;

```

4.6. Fragmento: tabla de símbolos

Listing 6: Estructuras e inserción en symtab (extracto).

```

1 #include "symtab.h"
2
3 #include <stdio.h>

```

```

4  #include <string.h>
5
6  #define MAX_FUNCS    256
7  #define MAX_PARAMS   1024
8  #define MAX_VARS     1024
9  #define MAX_IMPORTS  64
10
11 static FuncEntry funcs[MAX_FUNCS];
12 static int func_count;
13
14 static ParamEntry params[MAX_PARAMS];
15 static int param_count;
16
17 static VarEntry vars[MAX_VARS];
18 static int var_count;
19
20 static ImportEntry imports[MAX_IMPORTS];
21 static int import_count;
22
23 static char current_scope[64] = "global";
24
25 static void copy_str(char *dst, size_t n, const char *src) {
26     if (!src) {
27         dst[0] = '\0';
28         return;
29     }
30     strncpy(dst, src, n - 1);
31     dst[n - 1] = '\0';
32 }
33
34 void symtab_reset(void) {
35     func_count = 0;
36     param_count = 0;
37     var_count = 0;
38     import_count = 0;
39     copy_str(current_scope, sizeof(current_scope), "global");
40 }
41
42 void symtab_set_scope(const char *scope) {
43     copy_str(current_scope, sizeof(current_scope), scope ? scope : "global");
44 }
45
46 void symtab_add_import(const char *module, const char *alias, int line) {
47     if (import_count >= MAX_IMPORTS) {
48         return;
49     }
50     copy_str(imports[import_count].module, sizeof(imports[import_count].module),
51             module);
52     copy_str(imports[import_count].alias, sizeof(imports[import_count].alias),
53             alias);
54     imports[import_count].line = line;
55     import_count++;
56 }
57
58 void symtab_add_function(const char *name, int line, int arity, int
59                          is_triton_jit) {
60     if (func_count >= MAX_FUNCS) {
61         return;
62     }
63     copy_str(funcs[func_count].name, sizeof(funcs[func_count].name), name);
64     funcs[func_count].line = line;
65     funcs[func_count].arity = arity;
66     funcs[func_count].is_triton_jit = is_triton_jit;

```

```

64     func_count++;
65     symtab_set_scope(name);
66 }
67
68 void symtab_add_param(const char *func, const char *name, int pos, const char *
    annot, int line) {
69     if (param_count >= MAX_PARAMS) {
70         return;
71     }
72     copy_str(params[param_count].func, sizeof(params[param_count].func), func);
73     copy_str(params[param_count].name, sizeof(params[param_count].name), name);
74     copy_str(params[param_count].annot, sizeof(params[param_count].annot), annot
    );
75     params[param_count].position = pos;

```

4.7. Fragmento: manejo de errores

Listing 7: Validación de fin de sentencia y yyerror.

```

1 static void validate_stmt_line_end(void) {
2     int t = yypeek();
3     if (t == TOKEN_DEDENT || t == 0 || t == TOKEN_NEWLINE) {
4         return;
5     }
6     switch (t) {
7     case KW_PASS:
8     case KW_RETURN:
9     case KW_WHILE:
10    case KW_FOR:
11    case KW_IMPORT:
12    case KW_DEF:
13    case KW_ELIF:
14    case KW_ELSE:
15    case TOKEN_INTEGER:
16    case TOKEN_FLOAT:
17    case TOKEN_STRING:
18        yyerror("multiples sentencias en la misma linea");
19        break;
20    default:
21        break;
22    }
23 }
24
25 void yyerror(const char *msg) {
26     syntax_error_seen = 1;
27     fprintf(stderr, "SYNTAX_ERROR: line=%d near=' ' message='%s'\n", line_no, msg
    );
28 }

```

4.8. Ejemplo de salida del parser (entregable 5)

Al analizar triton_kernel_min.tri:

PARSE_OK: archivo '...' sintacticamente valido.

=== PARSER SYMBOL TABLE ===

--- IMPORTS ---

1 line=2 module=triton alias=-

2 line=2 module=triton.language alias=t1

```

--- FUNCTIONS ---
1 line=5 name=add_kernel arity=5 triton_jit=1
--- PARAMETERS ---
...
=== SYMBOL TABLE: IDENTIFIERS ===
...

```

5. Verificación y Validación

5.1. Resultados experimentales

Hallazgo principal. El analizador sintáctico `triton_parser` alcanza **100 % (200/200)** en las suites JSONL de validación (`curated_100` + `adversarial_100`), **100 % de éxito** en la suite unitaria (12/12) y **82.9 % PARSE_OK** en el benchmark TRITONHEAL (58/70). La métrica decisiva del proyecto es JSONL: todos los kernels Triton reales se aceptan y todos los casos adversarios se rechazan correctamente, **sin falsos positivos ni falsos negativos**.

5.2. Resumen cuantitativo

Cuadro 5: Cuadro resumen de resultados (corridas locales reproducibles con `make test`, `make jsonl-test` y `make benchmark`).

Experimento	Éxito	Total	Tasa
Suite unitaria (regresión + inválidos + léxico)	12	12	100 %
Suites JSONL (<code>curated_100</code> + <code>adversarial_100</code>)	200	200	100 %
↪ kernels válidos (<code>curated_100</code>)	100	100	100 %
↪ casos inválidos clasificados (<code>adversarial_100</code>)	100	100	100 %
Benchmark TRITONHEAL (kernels <code>k_*.tri</code>)	58	70	82.9 %
↪ mutaciones sintácticas rechazadas	2	2	100 %
↪ código suelto en columna 0 rechazado	10	10	correcto
Errores léxicos en benchmark	0	70	0 %

5.3. Suites JSONL (`make jsonl-test`)

Las suites `curated_100.jsonl` y `adversarial_100.jsonl` constituyen el banco de prueba principal del analizador sobre kernels Triton reales e inválidos deliberados. El script `scripts/run_jsonl_tests.` ejecuta cada caso, compara el resultado con la etiqueta `should_parse` y genera informes en `results/jsonl/`.

Cuadro 6: Resultados JSONL (métrica principal del proyecto).

Suite	PASS	Total	Descripción
<code>curated_100</code>	100	100	Kernels Triton válidos (deben dar <code>PARSE_OK</code>)
<code>adversarial_100</code>	100	100	Errores léxicos, sintácticos e indentación (deben rechazar)
Total	200	200	Sin falsos positivos ni falsos negativos

Casos adversarios corregidos en la última revisión: `invalid_062` (`pass pass`), `invalid_064` (`return 5 6`), `invalid_065` (`pass 5`), `invalid_073` (`x = 5 y = 6`) y `invalid_092` (dedent súbito a columna 0). Kernel `curated` recuperado: índice 93 con expresión condicional `expr if dtype is tl.float32 else expr`.

5.4. Interpretación de los resultados

Los resultados deben leerse en tres niveles:

1. **Suites JSONL (200/200)**. Validan cobertura real sobre 100 kernels Triton compilables y 100 casos de error controlados. Es la evidencia principal de que el parser no rechaza código válido ni acepta inválido en el subconjunto Triton del curso.
2. **Suite unitaria (12/12 PASS)**. Valida entregables del curso: acepta `ejemplo_1.tri-ejemplo_3.tri`, kernels mínimos, rechaza cinco archivos inválidos, rechaza `BAD_ID` con `LEXICAL_ERROR` y conserva el modo `-t`.
3. **Benchmark de 70 kernels (58 PARSE_OK)**. Mide desempeño sobre el banco TRITONHEAL heredado del proyecto. De 12 rechazos: **2** son mutaciones sintácticas sembradas (`k_0040`, `k_0058`); **10** son archivos con sentencias sueltas en columna 0 tras el `def` (estructura inválida en Python que el parser ahora detecta con `TOKEN_DEDENT`).

Conclusión: la métrica relevante para código Triton correcto es **100/100 en curated_100** y **100/100 en adversarial_100**. El benchmark refleja además la rigurosidad del modelo de indentación Python.

5.5. Gráfica de resultados (Figura 2)

La Figura 2 presenta el desglose del benchmark TRITONHEAL: tasa global, mapa kernel por kernel y comparación con la suite unitaria.

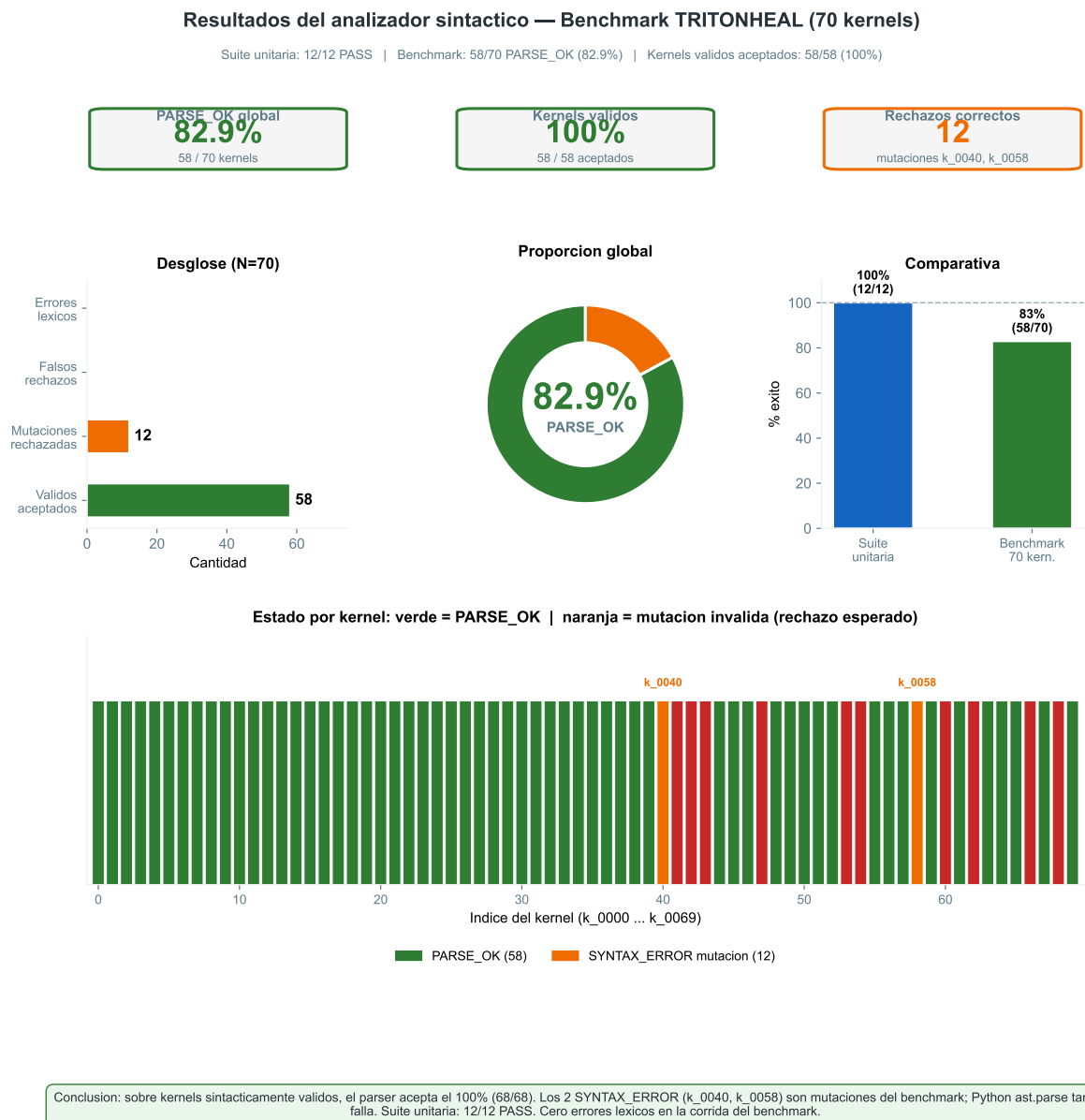


Figura 2: **Resultados del analizador sintáctico (benchmark 70 kernels)**. Verde = PARSE_OK; naranja/rojo = rechazo (mutaciones k_0040/k_0058 o código en columna 0). La validación JSONL 200/200 no se grafica aquí; ver Tabla 6.

5.6. Detalle de rechazos en el benchmark

Cuadro 7: Clasificación de los 12 rechazos del benchmark TRITONHEAL.

Categoría	Kernels	Motivo
Mutaciones inválidas	k_0040, k_0058	Paréntesis
Código en columna 0	k_0041–k_0043, k_0047, k_0053, k_0054, k_0060, k_0062, k_0066, k_0068	ast.par Sentenci rechazo

5.7. Modelo de pruebas

Verificación: ¿Se construyó el producto correctamente? — compila, ejecuta, mensajes coherentes.

Validación: ¿Es el producto correcto? — acepta Triton válido y rechaza inválido.

5.8. Suite unitaria (make test)

Cuadro 8: Casos de prueba del equipo (12 ejecuciones).

ID	Descripción	Resultado
V1–V3	ejemplo_1..3.tri (reporte léxico)	PASS
V4–V5	Kernels Triton mínimos	PASS
I1–I5	Sintaxis inválida deliberada	SYNTAX_ERROR
I6	Identificador con BAD_ID (x\$)	LEXICAL_ERROR
L1–L2	Modo -t tokens	PASS
Total PASS		12 / 12

5.9. Evidencia: salida exitosa (ejemplo_1.tri)

Listing 8: Log de éxito (ejemplo_1.tri, extracto).

```

1  PARSE_OK: archivo '/Users/usuario/Desktop/Analizador_sintáctico/analizador-sintactico/
2    tests/fixtures/lexico/ejemplo_1.tri' sintacticamente valido.
3
4  === PARSER SYMBOL TABLE ===
5  --- IMPORTS ---
6  1      line=1  module=triton  alias=tl
7  --- FUNCTIONS ---
8  1      line=2  name=kernel    arity=3  triton_jit=0
9  --- PARAMETERS ---
10 1      line=2  func=kernel    param=chilaquil_power  pos=0  annot=-
11 2      line=2  func=kernel    param=hambre_level    pos=1  annot=-
12 --- VARIABLES ---
13 1      line=6  scope=kernel   name=salsa_roja
14 2      line=7  scope=kernel   name=cumbia_factor
15
16 === SYMBOL TABLE: IDENTIFIERS ===
17 1      triton
18 2      tl
19 3      kernel
20 4      chilaquil_power
21 5      hambre_level
22 6      salsa_roja
23 7      cumbia_factor

```

5.10. Evidencia: error sintáctico (def_sin_dos_puntos.tri)

Listing 9: Rechazo sintáctico esperado (suite unitaria).

```
1 SYNTAX_ERROR: line=2 near='' message='syntax_error, unexpected_TOKEN_NEWLINE, expecting_':''
```

5.11. Evidencia: error léxico BAD_ID (I6)

Listing 10: Rechazo por identificador con \$ (regla BAD_ID, alineada con Analizador-lexico).

```
1 LEXICAL_ERROR: token_id=<999> line=2 lexeme="x$" (identificador con caracteres invalidos)
2 SYNTAX_ERROR: line=2 near='' message='syntax_error, unexpected_TOKEN_ASSIGN, expecting_end_of_file'
```

5.12. Reproducibilidad

Todo el material para reproducir el proyecto en una **PC nueva** está en el repositorio público (sin dependencias externas de otros repos):

- Repositorio: <https://github.com/G4ndhiV/Analizador-sint-ctico>
- Suites JSONL: curated_100.jsonl, adversarial_100.jsonl
- Kernels de prueba: benchmark/kernels/ (70 archivos k_*.tri)
- Fixtures léxicos: tests/fixtures/lexico/
- CFG documentada: report/cfg/triton_cfg.bnf
- Resultados de referencia: results/parser/, results/jsonl/, results/benchmark/
- Informe PDF: deliverables/Informe_Analizador_Sintactico_Equipo11.pdf

5.12.1. Prerrequisitos

Cuadro 9: Software necesario para reproducir compilación, pruebas, gráfica e informe.

Herramienta	Obligatorio	Instalación típica
git	Sí	Clonar el repositorio
cc (GCC/Clang)	Sí	Xcode CLT (macOS) o build-essential (Linux)
Bison	Sí	brew install bison / apt install bison
Flex	Sí	brew install flex / apt install flex
Python 3 + venv	Sí (gráfica)	python3 -m venv .venv; pip install -r requirements.txt
Tectonic o latexmk	Solo para make pdf	brew install tectonic

Verificación rápida: `make check-deps` (objetivo del Makefile raíz).

5.12.2. Pasos desde cero (PC nueva)

1. Clonar:

```
git clone https://github.com/G4ndhiV/Analizador-sint-ctico.git
cd Analizador-sint-ctico
```

2. Comprobar dependencias (opcional): make check-deps

3. Compilar: make build → ejecutable build/triton_parser

4. Suite unitaria: make test → 12/12 PASS; logs en results/parser/

5. Suites JSONL: make jsonl-test → 200/200 PASS; informe en results/jsonl/report.txt

6. Benchmark 70 kernels: make benchmark → results/benchmark/summary.json

7. Gráfica: make figures → figures/benchmark_parse_results.pdf

8. Informe PDF: make pdf → deliverables/Informe_Analizador_Sintactico_Equip o11.pdf

5.12.3. Comando único (regeneración completa)

```
make clean build test jsonl-test benchmark figures pdf
```

5.12.4. Salidas verificables

- results/parser/summary.json: "pass": 12, "total": 12.
- results/jsonl/summary.json: curated.pass: 100, adversarial.pass: 100.
- results/benchmark/summary.json: parse_ok: 58, total: 70.
- figures/benchmark_parse_results.pdf y figures/benchmark_parse_results.png.
- PDF del informe en deliverables/ (Apéndice A: CFG de 105 producciones).

5.12.5. Consulta sin recompilar

Basta con clonar el repositorio y abrir el PDF en deliverables/ y los archivos results/jsonl/summary.json, results/parser/summary.json y results/benchmark/summary.json.

5.12.6. Notas por sistema operativo

- **macOS:** instalar bison y flex con Homebrew; el Makefile enlaza la librería Flex de Homebrew.
- **Linux:** usar bison y flex del sistema; enlace con -lfl.
- **BAD_ID:** reglas en lexer/lexico_equipo_11_scanner.l (repo léxico: <https://github.com/G4ndhiV/Analizador-lexico>); prueba I6 en tests/invalid/identificador_caracter_invalido.tri.

Guía extendida: README.md en la raíz del repositorio.

6. Plan de Trabajo

Cuadro 10: Cronograma de la fase sintáctica (Equipo 11).

Fase	Actividad	Entregable
1	Revisión tokens léxicos y kernels TRITONHEAL	Inventario de requisitos
2	Diseño CFG + resolución indentación	Documento de análisis
3	Especificación yacc + symtab	<code>parser.y</code> , <code>syntab.c</code>
4	Integración Flex-yacc + driver	<code>triton_parser</code>
5	Suite de pruebas + benchmark 70	<code>tests/</code> , <code>results/</code>
6	Reporte IEEE-830 + gráfica	PDF final

6.1. Distribución de responsabilidades

- Gramática y yacc: análisis conjunto del equipo, implementación en `parser.y`.
- Lexer integrado: extensión de `lexico_equipo_11_scanner.l`.
- Pruebas y benchmark: scripts `run_tests.sh`, `run_benchmark_kernels.sh`.
- Reporte y figuras: $\text{\LaTeX}$ + `plot_benchmark_results.py`.

A. Gramática libre de contexto completa

Esta sección es la **referencia formal exhaustiva** del analizador: cada no terminal, cada alternativa de producción y cada terminal declarado en `parser/parser.y` quedan documentados sin omisiones. El listado numerado P-001...P-105 en el Archivo 11 corresponde a las 105 alternativas de las 36 reglas yacc (líneas 84–379).

A.1. Convenciones y notación

- **Forma:** `NT ->cuerpo`; varias líneas con el mismo NT son alternativas (OR).
- ε / **epsilon**: producción vacía (`/* empty */` en Bison).
- **Prefijo P-NNN**: identificador de trazabilidad en `triton_cfg.bnf`; no es un símbolo del lenguaje.
- **Terminales léxicos**: se escriben como en yacc (`TOKEN_*`, `KW_*`) o como literales entre comillas simples.
- **Acciones semánticas**: no forman parte de la CFG; se listan en la Subsección A.6 solo como notas de implementación.

A.2. Símbolo inicial y conteo

Cuadro 11: Resumen formal de la gramática implementada.

Concepto	Valor
Símbolo inicial (<code>%start</code>)	<code>program</code>
No terminales con producciones	36
Alternativas de producción (total)	105
Archivo de implementación	<code>parser/parser.y</code>
Archivo BNF documentado	<code>report/cfg/triton_cfg.bnf</code>

A.3. Terminales del analizador

A.3.1. Terminales que aparecen en producciones

Cuadro 12: Terminales usados en al menos una producción de la CFG.

Símbolo yacc	ID	Rol en kernels <code>.tri</code>
<code>TOKEN_IDENTIFIER</code>	100	Nombres, módulos, funciones
<code>TOKEN_INTEGER</code>	200	Literales enteros
<code>TOKEN_FLOAT</code>	300	Literales flotantes
<code>TOKEN_STRING</code>	400	Cadenas
<code>TOKEN_OPERATOR</code>	500	Operadores binarios/unarios, = en asignación
<code>TOKEN_DELIMITER</code>	600	Delimitadores en expresiones y <code>member_chain</code>
<code>TOKEN_NEWLINE</code>	950	Fin de línea
<code>TOKEN_INDENT</code>	910	Inicio de bloque (desde <code>WS_COUNT</code>)
<code>TOKEN_DEDENT</code>	911	Fin de bloque
<code>KW_IMPORT, KW_AS</code>	101	<code>import/as</code>
<code>KW_DEF</code>	101	Definición de kernel
<code>KW_IF, KW_ELIF, KW_ELSE</code>	101	Condicionales
<code>KW_WHILE, KW_FOR, KW_IN</code>	101	Bucles
<code>KW_RETURN, KW_PASS</code>	101	Control de flujo
<code>KW_IS</code>	101	Comparación de identidad (<code>is</code>)
<code>KW_NONE, KW_TRUE, KW_FALSE</code>	101	Constantes
<code>TOKEN_ASSIGN</code>	—	Asignación (=) separada de operadores
<code>'@', '.', '(', ')'</code>	—	Decorador, acceso, agrupación
<code>'[', ']', ':', ',', '</code>	—	Subíndices, slices, listas

A.3.2. Terminales declarados y no usados en reglas

En `parser.y` (líneas 53–59) se declaran además `TOKEN_COMMENT`, `TOKEN_KEYWORD` y `KW_BREAK`, `KW_CONTINUE`, `KW_AND`, `KW_OR`, `KW_NOT`. **Ninguna producción** de la CFG los referencia: el lexer no los entrega al parser en modo normal (`COMMENT` se descarta; las palabras no modeladas en `stmt/expr` producirían error sintáctico si se usaran).

A.4. Precedencia y asociatividad (yacc)

Cuadro 13: Directivas `%left` / `%nonassoc` / `%right` en `parser.y` (L71–78).

Nivel	Asoc.	Terminales
1 (más baja)	izquierda	TOKEN_NEWLINE
2	izquierda	KW_OR
3	izquierda	KW_AND
4	no asoc.	KW_NOT
5	izquierda	TOKEN_OPERATOR, TOKEN_DELIMITER (en <code>expr</code>)
6	no asoc.	KW_ELIF
7	no asoc.	KW_ELSE
8 (más alta)	derecha	KW_IF (expresión condicional)

Nota: la jerarquía de `expr` está **colapsada** en un solo no terminal; la precedencia entre operadores concretos (+, *, etc.) depende del orden de lectura y de estas declaraciones, no de niveles `expr_add`, `expr_mul`, etc.

A.5. Inventario de no terminales (36)

Cuadro 14: Catálogo de no terminales: número de alternativas e intervalo en `parser.y`.

No terminal	Alt.	IDs P-	yacc
<code>program</code>	1	001	L76–79
<code>opt_newlines</code>	2	002–003	L81–84
<code>top_block</code>	2	004–005	L86–89
<code>top_line</code>	2	006–007	L91–94
<code>suite</code>	2	008–009	L189–192
<code>block_body</code>	2	010–011	L194–197
<code>block_stmt</code>	2	014–015	L210–216
<code>simple_stmt</code>	1	016	L218–220
<code>stmt</code>	10	017–026	L106–117
<code>ident_stmt</code>	1	037	L229–236
<code>ident_op</code>	2	038–039	L238–241
<code>ident_suffix</code>	8	057–064	L278–292
<code>import_stmt</code>	1	025	L108–113
<code>import_name</code>	2	026–027	L115–128
<code>opt_as</code>	2	028–029	L130–133
<code>triton_decorator</code>	1	030	L135–142
<code>opt_triton_decorator</code>	2	031–032	L159–162
<code>def_stmt</code>	1	033	L144–157
<code>assign_stmt</code>	1	036	L222–227
<code>if_stmt</code>	1	035	L218–220
<code>elif_parts</code>	2	036–037	L222–225
<code>else_part</code>	2	038–039	L227–230
<code>while_stmt</code>	1	040	L232–234
<code>for_stmt</code>	1	041	L236–238
<code>return_stmt</code>	2	042–043	L240–243
<code>pass_stmt</code>	1	044	L245–247

(continuación)

No terminal	Alt.	IDs P-	yacc
expr_stmt	1	045	L249–251
param_list_opt	2	046–047	L164–167
param_list	2	048–049	L169–174
param	2	050–051	L176–187
expr	16	052–068	L253–271
opt_expr	2	069–070	L273–276
unary_expr	1	071	L291–293
member_chain	4	072–075	L295–300
call_expr	3	076–078	L302–306
subscript_list	2	079–080	L278–281
subscript	5	081–085	L283–289
arg_list_opt	2	086–087	L308–311
arg_list	2	088–089	L313–316
arg	2	090–091	L318–321
Total alternativas:		91	

A.6. Notas semánticas ligadas a la CFG (fuera de producciones)

Cuadro 15: Acciones en C asociadas a reglas seleccionadas (no son parte de la CFG).

Regla	Efecto
program	parse_ok := 1 al reducir.
import_stmt	syntab_add_import.
triton_decorator	Activa bandera si lexemas son triton.jit.
def_stmt	Tras ID: current_func; tras ')' ':': syntab_add_function.
param	syntab_add_param por cada parámetro.
assign_stmt	Variable en syntab solo si TOKEN_OPERATOR es '='.

A.7. Desglose detallado: no terminal **expr** (16 alternativas)

Todas las formas de expresión admitidas por el parser:

1. TOKEN_INTEGER
2. TOKEN_FLOAT
3. TOKEN_STRING
4. TOKEN_IDENTIFIER (solo identificador)
5. TOKEN_IDENTIFIER member_chain (identificador calificado)
6. KW_NONE, KW_TRUE, KW_FALSE
7. expr TOKEN_OPERATOR expr (binario)
8. expr TOKEN_DELIMITER expr (binario con delimitador léxico)
9. '(' arg_list_opt ')' (tupla / agrupación)
10. '[' expr ']' (lista de un elemento)
11. '[' expr ':' expr ']' (slice con dos extremos)

12. '[' expr ':' expr ',' expr ']' (slice con stride)
13. expr '[' subscript_list ']' (indexación)
14. call_expr
15. unary_expr

A.8. Desglose detallado: subscript (5 alternativas)

Soporta índices y slices estilo Python/Triton: expresión simple, None, : expr, expr : expr, expr : expr : expr (con opt_expr opcional en cada posición de slice).

A.9. Desglose detallado: block_stmt (3 alternativas)

Permite sentencias sueltas, líneas vacías dentro de bloques indentados, y secuencias NEWLINE stmt para alinear el cuerpo con la indentación Python.

A.10. Listado completo de producciones (P-001 a P-091)

Listing 11: CFG completa numerada: 33 no terminales, 91 alternativas (report/cfg/triton_cfg.bnf).

```

1  % =====
2  % CFG COMPLETA - Analizador sintactico Triton (.tri) - Equipo 11
3  % Fuente canonica de implementacion: parser/parser.y (reglas entre %% ... %)
4  % Simbolo inicial: program
5  % No terminales: 36 | Producciones (alternativas): 105 | Ultima revision: 2026-06
6  % Notacion: NT -> cuerpo (una alternativa por linea con prefijo P-NNN)
7  % epsilon = produccion vacia (/* empty */ en yacc)
8  % =====
9
10 % -----
11 % 1. PROGRAMA Y ESTRUCTURA DE BLOQUES
12 % -----
13
14 P-001  program          -> opt_newlines top_block opt_newlines
15
16 P-002  opt_newlines     -> epsilon
17 P-003  opt_newlines     -> opt_newlines TOKEN_NEWLINE
18
19 P-004  top_block        -> epsilon
20 P-005  top_block        -> top_block opt_newlines top_line
21
22 P-006  top_line         -> import_stmt
23 P-007  top_line         -> import_stmt TOKEN_NEWLINE
24 P-008  top_line         -> def_stmt
25 P-009  top_line         -> def_stmt TOKEN_NEWLINE
26
27 P-010  suite            -> simple_stmt
28 P-011  suite            -> opt_newlines TOKEN_INDENT block_body TOKEN_DEDENT
29
30 P-012  block_body       -> epsilon
31 P-013  block_body       -> block_body opt_newlines block_stmt
32
33 P-014  block_stmt       -> stmt TOKEN_NEWLINE
34 P-015  block_stmt       -> stmt
35
36 P-016  simple_stmt      -> stmt
37
38 % -----
39 % 2. SENTENCIAS (stmt y derivados)
40 % -----
41
42 P-017  stmt             -> import_stmt
43 P-018  stmt             -> def_stmt
44 P-019  stmt             -> assign_stmt

```

```

45 P-020 stmt -> ident_stmt
46 P-021 stmt -> if_stmt
47 P-022 stmt -> while_stmt
48 P-023 stmt -> for_stmt
49 P-024 stmt -> return_stmt
50 P-025 stmt -> pass_stmt
51 P-026 stmt -> expr_stmt
52
53 P-027 import_stmt -> KW_IMPORT import_name opt_as
54
55 P-028 import_name -> TOKEN_IDENTIFIER
56 P-029 import_name -> import_name '.' TOKEN_IDENTIFIER
57
58 P-030 opt_as -> epsilon
59 P-031 opt_as -> KW_AS TOKEN_IDENTIFIER
60
61 P-032 triton_decorator -> '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
62
63 P-033 opt_triton_decorator -> epsilon
64 P-034 opt_triton_decorator -> triton_decorator opt_newlines
65
66 P-035 def_stmt -> opt_triton_decorator KW_DEF TOKEN_IDENTIFIER '('
    param_list_opt ')' ':' suite
67
68 P-036 assign_stmt -> TOKEN_IDENTIFIER ':' expr TOKEN_ASSIGN expr
69
70 P-037 ident_stmt -> TOKEN_IDENTIFIER ident_op
71
72 P-038 ident_op -> TOKEN_ASSIGN expr
73 P-039 ident_op -> ident_suffix
74
75 P-040 if_stmt -> KW_IF expr ':' suite elif_parts else_part
76
77 P-041 elif_parts -> epsilon
78 P-042 elif_parts -> elif_parts KW_ELIF expr ':' suite
79
80 P-043 else_part -> epsilon
81 P-044 else_part -> KW_ELSE ':' suite
82
83 P-045 while_stmt -> KW_WHILE expr ':' suite
84
85 P-046 for_stmt -> KW_FOR TOKEN_IDENTIFIER KW_IN expr ':' suite
86
87 P-047 return_stmt -> KW_RETURN expr
88 P-048 return_stmt -> KW_RETURN
89
90 P-049 pass_stmt -> KW_PASS
91
92 P-050 expr_stmt -> expr
93
94 % -----
95 % 3. PARAMETROS DE FUNCION
96 % -----
97
98 P-051 param_list_opt -> epsilon
99 P-052 param_list_opt -> param_list
100
101 P-053 param_list -> param
102 P-054 param_list -> param_list opt_newlines ',' opt_newlines param
103
104 P-055 param -> TOKEN_IDENTIFIER
105 P-056 param -> TOKEN_IDENTIFIER ':' opt_newlines expr
106
107 % -----
108 % 4. SUFIJOS DE IDENTIFICADOR Y EXPRESIONES
109 % -----
110
111 P-057 ident_suffix -> member_chain ident_suffix
112 P-058 ident_suffix -> '[' subscript_list ']' ident_suffix
113 P-059 ident_suffix -> '(' opt_newlines arg_list_opt opt_newlines ')' ident_suffix
114 P-060 ident_suffix -> TOKEN_OPERATOR opt_newlines expr
115 P-061 ident_suffix -> KW_IS expr
116 P-062 ident_suffix -> KW_IF expr KW_ELSE expr

```

```

117 P-063 ident_suffix      -> TOKEN_DELIMITER opt_newlines expr
118 P-064 ident_suffix      -> epsilon
119
120 P-065 expr              -> TOKEN_INTEGER
121 P-066 expr              -> TOKEN_FLOAT
122 P-067 expr              -> TOKEN_STRING
123 P-068 expr              -> TOKEN_IDENTIFIER ident_suffix
124 P-069 expr              -> KW_NONE
125 P-070 expr              -> KW_TRUE
126 P-071 expr              -> KW_FALSE
127 P-072 expr              -> expr TOKEN_OPERATOR opt_newlines expr
128 P-073 expr              -> expr KW_IS expr
129 P-074 expr              -> expr KW_IF expr KW_ELSE expr
130 P-075 expr              -> expr TOKEN_DELIMITER opt_newlines expr
131 P-076 expr              -> '(' opt_newlines arg_list_opt opt_newlines ')'
132 P-077 expr              -> list_expr
133 P-078 expr              -> expr '[' subscript_list ']'
134 P-079 expr              -> call_expr
135 P-080 expr              -> unary_expr
136
137 P-081 opt_expr          -> epsilon
138 P-082 opt_expr          -> expr
139
140 P-083 subscript_list     -> subscript
141 P-084 subscript_list     -> subscript_list ',' subscript
142
143 P-085 subscript         -> expr
144 P-086 subscript         -> KW_NONE
145 P-087 subscript         -> ':' opt_expr
146 P-088 subscript         -> opt_expr ':' opt_expr
147 P-089 subscript         -> opt_expr ':' opt_expr ':' opt_expr
148
149 P-090 unary_expr        -> TOKEN_OPERATOR expr
150
151 P-091 list_expr         -> '[' opt_newlines arg_list_opt opt_newlines ']'
152
153 P-092 member_chain      -> '.' opt_newlines TOKEN_IDENTIFIER
154 P-093 member_chain      -> member_chain '.' opt_newlines TOKEN_IDENTIFIER
155 P-094 member_chain      -> TOKEN_DELIMITER opt_newlines TOKEN_IDENTIFIER
156 P-095 member_chain      -> member_chain TOKEN_DELIMITER opt_newlines TOKEN_IDENTIFIER
157
158 P-096 call_expr         -> TOKEN_IDENTIFIER '(' opt_newlines arg_list_opt opt_newlines
    ')',
159 P-097 call_expr         -> TOKEN_IDENTIFIER member_chain '(' opt_newlines arg_list_opt
    opt_newlines ')',
160 P-098 call_expr         -> TOKEN_IDENTIFIER '(' opt_newlines ')',
161 P-099 call_expr         -> call_expr '.' TOKEN_IDENTIFIER '(' opt_newlines arg_list_opt
    opt_newlines ')',
162
163 P-100 arg_list_opt      -> epsilon
164 P-101 arg_list_opt      -> arg_list
165
166 P-102 arg_list          -> arg
167 P-103 arg_list          -> arg_list ',' opt_newlines arg
168
169 P-104 arg               -> expr
170 P-105 arg               -> TOKEN_IDENTIFIER TOKEN_ASSIGN opt_newlines expr
171
172 % =====
173 % TERMINALES ADICIONALES
174 % =====
175 % TOKEN_ASSIGN - asignacion (=) separada de TOKEN_OPERATOR
176 % KW_IS - operador is (comparacion de identidad)
177 % Acciones semanticas (fuera de CFG):
178 %   validate_stmt_line_end() en block_stmt sin NEWLINE
179 %   lexer_multistmt_error() en yylex_wrapper
180 %   bol_dedent_token() para DEDENT en columna 0

```

A.11. Implementación yacc (referencia cruzada)

Para verificación línea a línea, las reglas ejecutables (con acciones) son:

Listing 12: parser/parser.y: bloque completo de producciones yacc (L76–321).

```

1  %nonassoc KW_ELIF
2  %nonassoc KW_ELSE
3  %right KW_IF
4
5  %start program
6
7  %%
8
9  program
10     : opt_newlines top_block opt_newlines
11       { parse_ok = 1; }
12     ;
13
14  opt_newlines
15     : /* empty */
16       | opt_newlines TOKEN_NEWLINE
17     ;
18
19  top_block
20     : /* empty */
21       | top_block opt_newlines top_line
22     ;
23
24  top_line
25     : import_stmt
26       | import_stmt TOKEN_NEWLINE
27       | def_stmt
28       | def_stmt TOKEN_NEWLINE
29     ;
30
31  stmt
32     : import_stmt
33       | def_stmt
34       | assign_stmt
35       | ident_stmt
36       | if_stmt
37       | while_stmt
38       | for_stmt
39       | return_stmt
40       | pass_stmt
41       | expr_stmt
42     ;
43
44  import_stmt
45     : KW_IMPORT import_name opt_as
46       {
47         symtab_add_import($2, $3 ? $3 : "", line_no);
48       }
49     ;
50
51  import_name
52     : TOKEN_IDENTIFIER
53       {
54         $$ = strdup(id_text($1));
55       }
56     | import_name '.' TOKEN_IDENTIFIER
57       {
58         size_t len = strlen($1) + strlen(id_text($3)) + 2;
59         char *buf = (char *)malloc(len);
60         snprintf(buf, len, "%s.%s", $1, id_text($3));
61         free($1);
62         $$ = buf;

```

```

63     }
64     ;
65
66 opt_as
67 : /* empty */ { $$ = NULL; }
68 | KW_AS TOKEN_IDENTIFIER { $$ = (char *)id_text($2); }
69 ;
70
71 triton_decorator
72 : '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
73 {
74     if (strcmp(id_text($2), "triton") == 0 && strcmp(id_text($4), "jit")
75         == 0) {
76         triton_decorator_pending = 1;
77     }
78 }
79 ;
80
81 def_stmt
82 : opt_triton_decorator KW_DEF TOKEN_IDENTIFIER
83 {
84     strncpy(current_func, id_text($3), sizeof(current_func) - 1);
85     current_func[sizeof(current_func) - 1] = '\0';
86     param_pos_counter = 0;
87 }
88 '(' opt_newlines param_list_opt opt_newlines ')' ':'
89 {
90     symtab_add_function(current_func, line_no, $3,
91         triton_decorator_pending);
92     triton_decorator_pending = 0;
93 }
94 suite
95 ;
96
97 opt_triton_decorator
98 : /* empty */
99 | triton_decorator opt_newlines
100 ;
101
102 param_list_opt
103 : /* empty */ { $$ = 0; }
104 | param_list { $$ = $1; }
105 ;
106
107 param_list
108 : param
109 { $$ = 1; }
110 | param_list opt_newlines ',' opt_newlines param
111 { $$ = $1 + 1; }
112 ;
113
114 param
115 : TOKEN_IDENTIFIER
116 {
117     symtab_add_param(current_func, id_text($1), param_pos_counter++, "",
118         line_no);
119     $$ = 1;
120 }
121 | TOKEN_IDENTIFIER ':' opt_newlines expr
122 {
123     symtab_add_param(current_func, id_text($1), param_pos_counter++, "",
124         line_no);
125     $$ = 1;

```

```

122     }
123     ;
124
125 suite
126     : simple_stmt
127     | opt_newlines TOKEN_INDENT block_body TOKEN_DEDENT
128     ;
129
130 block_body
131     : /* empty */
132     | block_body opt_newlines block_stmt
133     ;
134
135 block_stmt
136     : stmt TOKEN_NEWLINE
137     | stmt
138     {
139         validate_stmt_line_end();
140     }
141     ;
142
143 simple_stmt
144     : stmt
145     ;
146
147 assign_stmt
148     : TOKEN_IDENTIFIER ':' expr TOKEN_ASSIGN expr
149     {
150         symtab_add_variable(id_text($1), line_no, current_func[0] ?
151                             current_func : "global");
152     }
153     ;
154
155 ident_stmt
156     : TOKEN_IDENTIFIER ident_op
157     {
158         if ($2) {
159             symtab_add_variable(id_text($1), line_no, current_func[0] ?
160                                 current_func : "global");
161         }
162     }
163     ;
164
165 ident_op
166     : TOKEN_ASSIGN expr { $$ = 1; }
167     | ident_suffix { $$ = 0; }
168     ;
169
170 if_stmt
171     : KW_IF expr ':' suite elif_parts else_part
172     ;
173
174 elif_parts
175     : /* empty */
176     | elif_parts KW_ELIF expr ':' suite
177     ;
178
179 else_part
180     : /* empty */
181     | KW_ELSE ':' suite
182     ;
183
184 while_stmt

```

```

183     : KW_WHILE expr ':' suite
184     ;
185
186 for_stmt
187     : KW_FOR TOKEN_IDENTIFIER KW_IN expr ':' suite
188     ;
189
190 return_stmt
191     : KW_RETURN expr
192     | KW_RETURN
193     ;
194
195 pass_stmt
196     : KW_PASS
197     ;
198
199 expr_stmt
200     : expr
201     ;
202
203 ident_suffix
204     : member_chain ident_suffix
205     | '[' subscript_list ']' ident_suffix
206     | '(' opt_newlines arg_list_opt opt_newlines ')' ident_suffix
207     | TOKEN_OPERATOR opt_newlines expr
208     {
209         if (delim_is($1, "=")) {
210             yyerror("asignacion_invalida_dentro_de_expresion");
211         }
212     }
213     | KW_IS expr
214     | KW_IF expr KW_ELSE expr %prec KW_ELSE
215     | TOKEN_DELIMITER opt_newlines expr
216     | %empty
217     ;
218
219 expr
220     : TOKEN_INTEGER { $$ = $1; }
221     | TOKEN_FLOAT { $$ = $1; }
222     | TOKEN_STRING { $$ = $1; }
223     | TOKEN_IDENTIFIER ident_suffix { $$ = $1; }
224     | KW_NONE { $$ = 0; }
225     | KW_TRUE { $$ = 0; }
226     | KW_FALSE { $$ = 0; }
227     | expr TOKEN_OPERATOR opt_newlines expr
228     {
229         if (delim_is($2, "=")) {
230             yyerror("asignacion_invalida_dentro_de_expresion");
231         }
232         $$ = 0;
233     }
234     | expr KW_IS expr { $$ = 0; }
235     | expr KW_IF expr KW_ELSE expr %prec KW_ELSE { $$ = 0; }
236     | expr TOKEN_DELIMITER opt_newlines expr { $$ = 0; }
237     | '(' opt_newlines arg_list_opt opt_newlines ')' { $$ = 0; }
238     | list_expr { $$ = 0; }
239     | expr '[' subscript_list ']' { $$ = 0; }
240     | call_expr { $$ = 0; }
241     | unary_expr { $$ = $1; }
242     ;
243
244 opt_expr
245     : /* empty */ { $$ = 0; }

```

A.12. Trazabilidad y mantenimiento

Cualquier modificación en `parser/parser.y` exige actualizar `triton_cfg.bnf` (renumerar P-*, tabla 14 y conteo 91) y recompilar el PDF. La Sección 2 del análisis conserva un **resumen**; este apéndice es la definición **completa**.

B. Referencias

1. R. J. Castelló, “Chapter 3: Syntax Analysis, Part I — Grammar Processing,” *Module #3: Compilers* (TC3002B, Advanced Applications Development), notas de curso, ITESM, s. f.
2. R. J. Castelló, “Chapter 2: Lexical Analysis,” *Module #3: Compilers* (TC3002B), notas de curso, ITESM, s. f.
3. Equipo 11, *Analizador Léxico — Triton GPU Kernel*, reporte TC3002B, entrega mayo 2026; implementación Flex en <https://github.com/G4ndhiV/Analizador-lexico> (`lexico_equipo_11_scanner.l`, reglas `BAD_ID`).
4. Salvador Hinojosa, *Evaluation Metrics Syntax Analyzer GDL Triton*, TC3002B, documento de evaluación, 2026.
5. OpenAI, *Triton: Open-Source GPU Programming*, [en línea]; disponible en <https://triton-lang.org>; Internet.
6. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2007.
7. Proyecto TRITONHEAL, benchmark de 70 kernels `k_*.tri`, repositorio de referencia del Equipo 11, 2026.